



APOLLO INSTITUTE OF COMPUTER APPLICATION

BACHELOR OF COMPUTER APPLICATIONS

ADVANCED WEB TECHNOLOGY (BC02001031)

ASSIGNMENT - UNIT 5

Short Answer Questions

1. What is Angular?
Ans: Angular (web framework) is a popular open-source front-end web application framework developed and maintained by Google. It is used for building dynamic, single-page web applications (SPA). Angular allows developers to create interactive and scalable web applications using technologies such as HTML, CSS, and TypeScript. It follows a component-based architecture, which makes applications easier to develop, maintain, and test.
2. Define component in Angular.
Ans: A component is the fundamental building block of an Angular application. It controls a specific part of the user interface (UI) and defines how the view should be displayed and behave. Each component consists of three main parts: a **template** (HTML code for the UI), a **class** (written in TypeScript to handle logic), and **metadata** (which describes how the component should be processed). Components help in dividing the application into smaller reusable parts.
3. What is a module in Angular?
Ans: A module in Angular is a logical container that groups related components, directives, pipes, and services together. Modules help organize the application into cohesive blocks of functionality. Every Angular application has at least one root module called **AppModule**, which is responsible for bootstrapping the application. Modules improve code maintainability, readability, and scalability by structuring the application effectively.
4. What is a selector in Angular?
Ans: A selector in Angular is a property defined in the component's metadata that specifies the HTML tag used to include the component in a template. It acts as a custom HTML element representing the component. When Angular encounters this selector in HTML, it renders the corresponding component. For example, if the selector is **app-root**, the component can be used in HTML as `<app-root></app-root>`.
5. What is bootstrapping in Angular?
Ans: Bootstrapping is the process of starting or initializing an Angular application. During bootstrapping, Angular loads the root module and creates the root component, which serves as the entry point of the application. This process connects the Angular application with the HTML page and prepares it to run in the browser.
6. List some IDEs used for Angular development.
Ans: Several Integrated Development Environments (IDEs) and code editors can be used for Angular development. Popular ones include Visual Studio Code, WebStorm, Sublime Text, and Atom (text editor). These IDEs provide useful features such as syntax highlighting, debugging tools, extensions, and integrated terminal support, which make Angular development easier and more efficient.
7. What is TypeScript?

Ans: TypeScript is an object-oriented programming language developed by Microsoft. It is a superset of JavaScript, which means all JavaScript code is valid TypeScript code. TypeScript adds additional features such as static typing, interfaces, classes, and modules. These features help developers write more structured, maintainable, and error-free code, which is why Angular uses TypeScript as its primary programming language.

8. List different data types in TypeScript.

Ans: TypeScript supports several data types that help define the type of values stored in variables. Some commonly used data types include **number**, **string**, **boolean**, **array**, **tuple**, **enum**, **any**, **void**, **null**, and **undefined**. These data types help improve code reliability by enabling type checking during development.

9. What is an enum in TypeScript?

Ans: An enum (short for enumeration) is a special data type in TypeScript that allows developers to define a set of named constants. Enums make the code more readable and easier to maintain by assigning meaningful names to numeric values. For example, days of the week or status codes can be represented using enums instead of using plain numbers.

10. What is a lambda function?

Ans: A lambda function is an anonymous function that does not have a name and is typically used for short operations or expressions. In TypeScript and JavaScript, lambda functions are commonly written using arrow function syntax. They are widely used in callbacks, event handling, and functional programming patterns.

11. Define class in TypeScript.

Ans: A class in TypeScript is a blueprint used to create objects. It defines properties (variables) and methods (functions) that describe the behavior and characteristics of an object. Classes support object-oriented programming concepts such as encapsulation, inheritance, and polymorphism. They help organize code and make applications more modular and reusable.

12. What is an interface in TypeScript?

Ans: An interface in TypeScript is used to define the structure or blueprint of an object. It specifies the properties and methods that an object or class must implement. Interfaces help ensure consistency in the code and make it easier to manage large applications by defining clear contracts between different parts of the program.

13. What are decorators in TypeScript?

Ans: Decorators are special functions used in TypeScript to add metadata or modify the behavior of classes, methods, properties, or parameters. They are widely used in Angular to define components, services, modules, and other elements. Decorators help simplify configuration and provide additional functionality without modifying the core logic.

14. What is the purpose of @Component decorator?

Ans: The **@Component** decorator is used in Angular to define a class as a component. It provides metadata about the component, such as the selector (HTML tag name), template (HTML view), styles (CSS), and other configuration options. This metadata tells Angular how to create and use the component within the application.

15. Write the syntax of an arrow function.

Ans: 1. An arrow function is a concise way to write a function using the arrow ($\Rightarrow$) syntax in TypeScript and JavaScript. It is commonly used for short functions and callbacks.

General syntax:

(parameters) $\Rightarrow$ expression

Example:

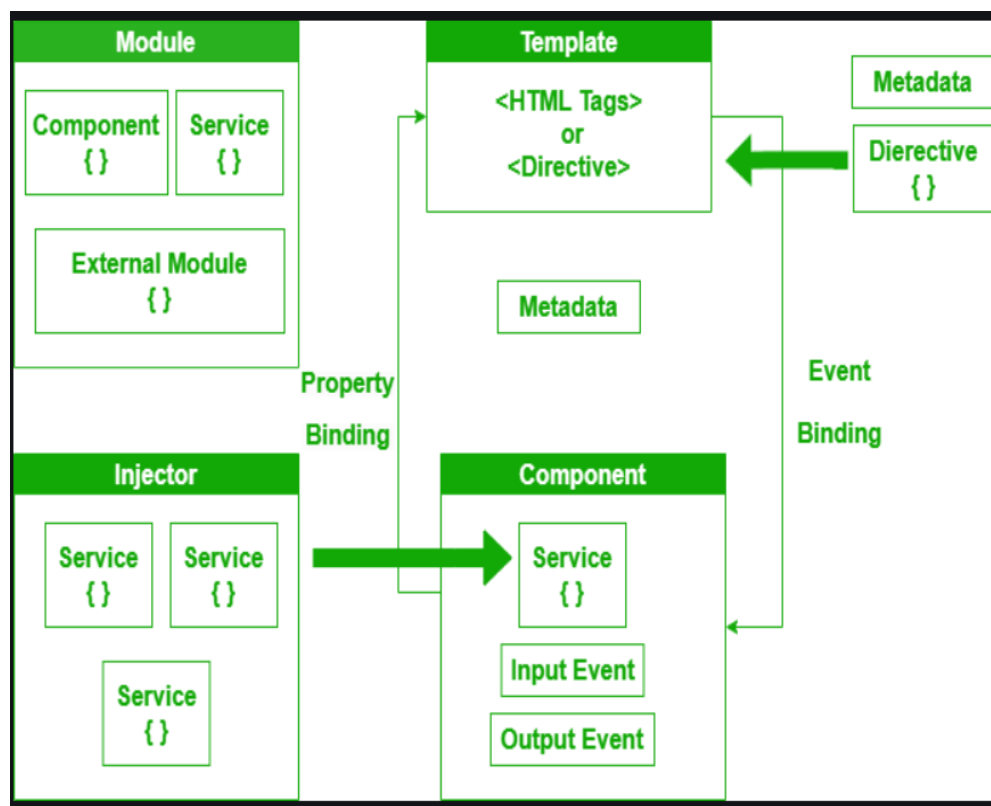
```
let add = (a: number, b: number) => {
  return a + b;
};
```

Arrow functions provide a shorter syntax and automatically bind the value of **this**, making them very useful in modern web development.

Long Answer Questions

1. Explain Angular architecture and its components

Ans: Angular is a client-side front-end framework developed by a team of developers at Google based on Typescript. It is used for building dynamic and single-page web applications (SPAs). Also, Angular is a versatile framework for building web applications and offers a wide range of features and tools to streamline the development process and create robust and maintainable applications.



Component of the Architecture:

1. Modules (NgModules)

- Angular apps are modular.
- A root module (AppModule) bootstraps the application.
- Feature modules help organize code logically.

2. Components

- The **core UI building blocks**.
- Each component consists of:
 - HTML template (view)
 - TypeScript class (logic)
 - CSS styles

Example:

```
@Component({
  selector:'app-root',
  templateUrl:'./app.component.html',
})
export class AppComponent { }
```

3. Templates

- Define the UI using HTML.
- Include Angular syntax like:
 - *ngIf, *ngFor
 - Interpolation {{ }}

4. Data Binding

Angular supports 4 types:

- Interpolation → {{data}}
- Property Binding → [property]="value"
- Event Binding → (event)="handler()"
- Two-way Binding → [(ngModel)]

5. Directives

- Extend HTML functionality.
- Types:
 - Structural (*ngIf, *ngFor)
 - Attribute (ngClass, ngStyle)

6. Services

- Used for **business logic and data sharing**.
- Promotes reusability.
- Example: API calls, authentication.

7. Dependency Injection (DI)

- Angular uses DI to provide services to components.
- Improves modularity and testability.

8. Routing

- Enables navigation between views.
- Uses Angular Router.

9. Pipes

- Transform data in templates.
- Example:

```
{{ today | date }}
```

2. Explain Angular components with an example

Ans: Angular is a **TypeScript-based open-source front-end web application framework** developed and maintained by the Google. It is used to build **dynamic, single-page web applications (SPA)**.

Angular provides:

- Component-based architecture
- Two-way data binding

- Dependency injection
- Powerful routing system

Features of Angular

- Modular structure
- Reusable components
- Faster development
- Easy testing and maintenance
- Supports TypeScript

A **Component** is the **basic building block of an Angular application**.

Each component controls a part of the user interface.

A component consists of:

1. **Template (HTML)** – View
2. **Class (TypeScript)** – Logic
3. **Metadata** – Configuration

Example

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-hello',
  template: '<h1>Hello Angular</h1>'
})
export class HelloComponent {
}
```

Explanation

- `@Component` → Decorator defining metadata
- `selector` → HTML tag used to display component
- `template` → HTML view

Usage in HTML:

```
<app-hello></app-hello>
```

3. Explain modules in Angular with suitable example

Ans: **Definition**

An Angular **Module** is a container for a group of related components, directives, pipes, and services.

Angular applications have at least **one module called the root module**.

Purpose of Modules

- Organize application into logical sections
- Enable code reusability
- Support lazy loading
- Improve maintainability and readability

Types of Angular Modules

1. Root Module

- The main module (AppComponent)
- Bootstraps the application

2. Feature Modules

- Used to organize specific functionality (e.g., Admin, User)

3. Shared Module

- Contains reusable components, pipes, directives

4. Core Module

- Contains singleton services used across the app

Example

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HelloComponent } from './hello.component';
```

```
@NgModule({
  declarations: [HelloComponent],
  imports: [BrowserModule],
  bootstrap: [HelloComponent]
})
export class AppModule { }
```

Explanation

- declarations → Components used in module
- imports → External modules
- bootstrap → Root component

4. Describe the concept of selector in Angular

Ans: **Introduction**

In Angular (developed by Google), a **selector** is a property of a component that defines how that component is **identified and used in HTML**.

It acts as a **custom HTML tag** through which Angular inserts and renders the component in the DOM.

Definition

A **selector** is a unique string defined in the `@Component` decorator that tells Angular **where and how to use the component** in an HTML template.

Syntax

```
@Component({
  selector: 'app-example',
  template: '<h1>Example Component</h1>'
})
```

Here, app-example is the selector.

Example of Selector Usage

Step 1: Component Definition

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-welcome',
  template: '<h2>Welcome to Angular!</h2>'
})
export class WelcomeComponent { }
```

Step 2: Using Selector in HTML

```
<app-welcome></app-welcome>
```

Working of Selector

1. Angular scans HTML templates
2. Finds the matching selector (`<app-welcome>`)

3. Replaces it with the component's template
4. Renders it in the browser

Types of Selectors in Angular

Angular supports different types of selectors:

1. Element Selector (Most Common)

- Used as an HTML tag
- Example:

selector: 'app-header'

Usage:

```
<app-header></app-header>
```

2. Attribute Selector

- Used as an HTML attribute
- Example:

selector: '[appHighlight]'

Usage:

```
<p appHighlight>Text</p>
```

3. Class Selector

- Applied as a CSS class
- Example:

selector: '.app-style'

Usage:

```
<div class="app-style"></div>
```

5. Explain bootstrapping process in Angular

Ans: **Introduction**

In Angular (developed by Google), **bootstrapping** is the process of initializing and launching an Angular application. It tells Angular **where the application starts and which component should be loaded first**.

Definition

Bootstrapping is the process of loading the **root module (AppModule)** and rendering the **root component** in the browser.

The root component is usually AppComponent.

Types of Bootstrapping

1. Automatic Bootstrapping (Browser-based apps)

- Used in most Angular applications
- Defined in main.ts

2. Manual Bootstrapping

- Used for advanced scenarios
- Developer explicitly controls startup

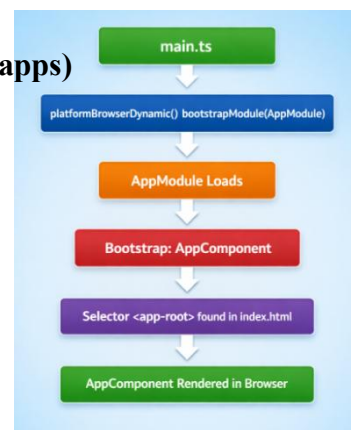
Bootstrapping Process (Step-by-Step)

Step 1: Entry Point (main.ts)

Angular starts execution from main.ts.

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
```

```
import { AppModule } from './app/app.module';
```



```
platformBrowserDynamic().bootstrapModule(AppModule)
    .catch(err => console.error(err));
```

Step 2: Root Module Loading

- Angular loads the **AppModule**
- This module contains all app configuration

Step 3: Bootstrap Component Declaration

Inside AppModule, Angular finds the bootstrap component:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Step 4: Root Component Execution

- Angular creates an instance of AppComponent
- Loads its template into the browser

Step 5: Rendering in DOM

The root component is inserted into:

```
<app-root></app-root>
```

This tag is usually present in index.html.

6. Explain TypeScript and its advantages

Ans: TypeScript is a **superset of JavaScript** developed by Microsoft.

It adds:

- Static typing
- Classes
- Interfaces
- Modern JavaScript features

TypeScript code is compiled into **JavaScript**.

Advantages

- Better error detection
- Object-oriented features
- Large application support
- Easy debugging

7. Explain different data types in TypeScript with examples

Ans: TypeScript provides several **data types**.

1. String

Used to store text values.

Example

```
let name: string = "Angular";
console.log(name);
```

2. Number

Stores numeric values.

Example

```
let age: number = 25;
console.log(age);
```

3. Boolean

Stores true or false values.

Example

```
let isActive: boolean = true;
```

4. Array

Stores multiple values.

Example

```
let numbers: number[] = [10,20,30,40];
```

Alternative syntax:

```
let numbers: Array<number> = [10,20,30];
```

5. Enum

Enum represents a group of named constants.

Example

```
enum Color {  
    Red,  
    Green,  
    Blue  
}
```

```
let c: Color = Color.Green;  
console.log(c);
```

6. Void

Void represents **no return value**.

Example

```
function display(): void {  
    console.log("Hello");  
}
```

8. Explain functions and lambda expressions in TypeScript

Ans: **Normal Function**

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

Lambda Function (Arrow Function)

Lambda functions provide shorter syntax.

```
let sum = (a: number, b: number): number => {  
    return a + b;  
};
```

9. Explain classes and interfaces in TypeScript with examples

Ans: **Classes in TypeScript**

Classes are used to create **objects and implement object-oriented programming**.

Example

```
class Person {  
    name: string;
```

```
constructor(name: string) {  
    this.name = name;  
}  
  
display() {  
    console.log(this.name);  
}  
}  
  
let p = new Person("Rahul");  
p.display();
```

Interface in TypeScript

An **Interface** defines the structure of an object.

Example

```
interface Student {  
    id: number;  
    name: string;  
}
```

```
let s: Student = {  
    id: 1,  
    name: "Amit"  
};
```

Interfaces ensure **type safety**.